Tecnologías, responsabilidades y motivos

Proyecto: Huellitas Saludables PRO
Fecha de revisión: 13 de noviembre de 2026
Propósito: material de referencia para estudiantes y equipo técnico.
Alcance: consolidación estática de las auditorías de frontend, backend, persistencia, seguridad, pruebas, documentación de API e infraestructura. No se modificó código ni se ejecutaron pruebas, builds, migraciones, contenedores o comandos destructivos.

> **Cómo leer los motivos.** «Documentado» significa que una fuente del repositorio expresa la decisión o el propósito. «Inferencia técnica» describe una explicación razonable a partir de la configuración/código, no una declaración histórica del equipo. «Histórico no registrado» indica que no se encontró evidencia de por qué se eligió originalmente una tecnología ni una comparación formal. No se presentan inferencias como hechos históricos.

1. Resumen ejecutivo del stack

Área/capa	Tecnología	Versión observada	Función resumida
Presentación/frontend	React	`^18.3.1` ; lock `18.3.1`	Componentes y páginas de la SPA.
Presentación/frontend	React DOM	`^18.3.1` ; lock `18.3.1`	Montaje de React en el DOM.
Presentación/frontend	TypeScript	`^5.7.2` ; lock `5.9.3`	Lenguaje y comprobación estática del frontend.
Presentación/frontend	Vite	`^6.4.3` ; lock `6.4.6` en `front/`	Desarrollo y build del frontend. Los manifiestos raíz tienen otra versión declarada: `^8.3.8` .
Presentación/frontend	`@vitejs/plugin-react`	`^4.3.4` ; lock `4.7.0`	Integración React para Vite.
Presentación/frontend	React Router DOM	`^7.18.4` ; lock `7.18.4`	Navegación SPA y guards de presentación.
Presentación/frontend	Axios	`^1.7.9` ; lock `1.20.0`	Cliente HTTP común hacia la API.
Presentación/frontend	React Icons	`^5.4.0` ; lock `5.7.0`	Iconos puntuales, comprobados en NuevaCita.
Presentación/frontend	CSS nativo	No declarada	Estilos globales propios; no se encontró framework UI/CSS.
Presentación/frontend	Google Fonts: DM Sans y Nunito	Sin versión de paquete	Tipografías remotas con fuentes de respaldo locales.
Frontend/build	`@types/react`	`^18.3.18` ; lock `18.3.31`	Tipos de desarrollo para React.
Frontend/build	`@types/react-dom`	`^18.3.5` ; lock `18.3.7`	Tipos de desarrollo para React DOM.
Frontend/build	`@types/node`	`^22.10.2` ; lock `22.20.4`	Tipos de Node para configuraciones/herramientas.
Backend/runtime	Node.js	22 (`node:22-alpine` ; parche no fijado)	Runtime de API JavaScript ESM.
Backend/lenguaje	JavaScript ESM	Estándar no declarado	Implementación de servidor, rutas, controladores, servicios

Área/capa	Tecnología	Versión observada	Función resumida
			y modelos.
Backend/API	Express	`^4.18.2` ; lock `4.22.3`	Framework HTTP, middlewares y rutas.
API/contrato	HTTP y JSON	Versiones no declaradas	Transporte y formato principal de la API.
Persistencia	PostgreSQL	16 (`postgres:16-alpine` ; parche/digest no fijados)	Motor relacional.
Persistencia	Prisma ORM, Client y CLI/Migrate	`^7.10.1` ; lock `7.10.1`	Esquema, cliente de acceso a datos y migraciones.
Persistencia	`@prisma/adapter-pg`	`^7.10.1` ; lock `7.10.1`	Adaptador Prisma para PostgreSQL.
Persistencia	`pg` (node-postgres)	`^8.23.0` ; lock `8.23.1`	Driver utilizado por el adapter y setup de pruebas.
Persistencia	SQL/PL/pgSQL de PostgreSQL	No declarada	CHECKs, triggers, índice parcial y consultas con bloqueos.
Seguridad	`jsonwebtoken` / JWT	`^9.0.3` ; lock `9.0.3`	Tokens Bearer HS256 y verificación de identidad.
Seguridad	`bcrypt`	`^6.0.0` ; lock `6.0.0`	Hashes actuales de contraseña y rehash progresivo.
Seguridad/compatibilidad	scrypt nativo de Node	API del runtime; no paquete	Verificación únicamente de hashes históricos.
Seguridad/validación	Joi	`^17.13.6` ; lock `17.13.9`	Esquemas de validación de solicitudes.
Seguridad/CORS	`cors`	`^2.8.5` ; lock `2.8.6`	Middleware de CORS en Express.
Seguridad/logging	Pino	`^9.14.0` ; lock `9.14.0`	Logging estructurado, niveles y redacción configurada.
Configuración	dotenv	`^16.3.1` ; lock `16.6.1`	Carga de variables de entorno.

Área/capa	Tecnología	Versión observada	Función resumida
Seguridad/SQL	Middleware propio y Prisma tagged raw SQL	Código propio / Prisma 7	Normalización/validación, consultas parametrizadas y controles de parámetros.
Fecha/hora	`Date`, `Intl.DateTimeFormat`	APIs nativas de Node	Regla horaria de negocio en `America/Bogota`.
Pruebas/backend	Vitest	`4.1.12`	Runner actual de pruebas backend.
Pruebas/backend	Supertest	`^7.2.2`; lock `7.3.2`	Solicitudes HTTP de integración contra Express.
Pruebas/cobertura	`@vitest/coverage-v8`	`4.1.12`	Reportes de cobertura V8.
Pruebas/herramientas	`cross-env`	`^10.1.0`; lock `10.1.0`	Define `NODE_ENV=test` en scripts.
Desarrollo backend	`tsx`	`^4.23.15`; lock `4.23.15`	Ejecuta scripts de arranque/mantenimiento/seed.
Desarrollo backend	nodemon	`^3.0.1`; lock `3.1.14`	Recarga el servidor en desarrollo.
Documentación API	OpenAPI	3.0.0 declarado	Contrato descriptivo, mantenido manualmente.
Documentación API	`swagger-jsdoc`	`^6.2.8`; lock `6.3.0`	Construye el objeto de especificación.
Documentación API	`swagger-ui-express`	`^5.0.0`; lock `5.0.1`	Publica Swagger UI en `/api-docs`.
Construcción/ejecución	npm	No fijada; lockfile v3	Instalación de dependencias y scripts.
Despliegue local	Docker	No declarada	Construcción y ejecución de imágenes.
Despliegue local	Docker Compose	README indica v2; YAML sin versión	Orquestación local y stack separado de pruebas.
Frontend/runtime	Nginx	`nginx:1.27-alpine`	Archivos SPA y proxy `/api`, `/health`, `/api-docs`.
Infraestructura	`/bin/sh` y `docker-entrypoint.sh`	Sin versión	Espera a PostgreSQL, migra, opcionalmente seed y ejecuta backend.

Área/capa	Tecnología	Versión observada	Función resumida
Calidad frontend	ESLint	No declarada	Un script intenta ejecutar `eslint .`, pero no declara dependencia local.

Los rangos `^` son los declarados en `package.json`; la resolución indicada proviene del lockfile cuando se verificó. No equivale a una versión de imagen inmutable. Los documentos de auditoría identifican ocho modelos/tablas actuales: `usuario`, `raza`, `mascota`, `usuario_mascota`, `disponibilidad`, `cita`, `historia_clinica` y `cita_cancelacion`.

2. Frontend: presentación, navegación y comunicación con API

2.1 React, React DOM y TypeScript

React — `^18.3.1` (lock `18.3.1`) | Capa: frontend/runtime. En general, biblioteca para construir interfaces mediante componentes. En el proyecto compone la SPA, su estado/contexto, páginas, formularios, dashboards y piezas reutilizables; arranca desde `src/main.tsx` con `React.StrictMode`. Los guards (`PrivateRoute`, `RoleRoute`) condicionan lo que se presenta, pero no reemplazan la autorización de la API.

- **Motivo documentado:** el documento maestro DT-02 recomienda mantener React y reutilizar las pantallas y patrones existentes.
- **Justificación técnica inferida:** la estructura de páginas/componentes/contexto facilita compartir UI y estado.
- **Motivación histórica:** no se encontró acta de elección original ni comparación contemporánea.
- **Beneficios y límites:** reutilización y composición de UI; no es router, cliente HTTP ni frontera de seguridad. No hay benchmark de frontend.
- **Evidencia:** `front/src/main.tsx`, `front/src/App.tsx`, `front/src/routes/AppRoutes.tsx`, `front/src/context/AuthContext.tsx`, `front/src/components/`, `front/src/pages/`, documento maestro, DT-02.

React DOM — `^18.3.1` (lock `18.3.1`) | Capa: frontend/runtime. En general, renderer que conecta React con el DOM del navegador. Aquí `createRoot` monta la aplicación en `#root` y entrega el árbol `StrictMode`/router/proveedor de autenticación. El maestro documenta React 18, pero no una decisión separada sobre React DOM; su uso se infiere del arranque. La razón histórica de selección no está registrada. Evidencia: `front/src/main.tsx`, `front/index.html`, `front/package.json` y `front/package-lock.json`.

TypeScript — `^5.7.2` (lock `5.9.3`) | Capa: frontend/build. Lenguaje tipado que se transforma/comprueba antes de ejecutar JavaScript. Se usa en la mayor parte de los archivos `.ts/.tsx`; `build` combina `tsc -b` y Vite, y existe `type-check` con `tsc -b --noEmit`. El maestro indica que ayuda a expresar contratos del frontend; inferencia técnica: los tipos hacen explícitos algunos datos y props. La elección histórica no está documentada. Limitación relevante: `tsconfig.json` mantiene `strict:false`, `noUnusedLocals:false` y `noUnusedParameters:false`; la comprobación no demuestra cobertura ni exactitud de todos los tipos. Evidencia: `front/tsconfig.json`, `front/tsconfig.node.json`, `front/package.json`, `front/src/vite-env.d.ts`.

Tipos de React/Node — `@types/react` `^18.3.18` (lock `18.3.31`), `@types/react-dom` `^18.3.5` (lock `18.3.7`) y `@types/node` `^22.10.2` (lock `22.20.4`) | Capa: frontend/build. Paquetes de declaraciones

para editor/compilador, no runtimes del navegador. Aportan tipos de React/DOM y Node, este último principalmente para configuración Vite. La inclusión se observa en manifiestos, pero no hay motivo histórico independiente documentado para cada paquete; la utilidad se infiere de TypeScript y sus APIs. No se comprobó si cada tipo Node es estrictamente necesario. Evidencia: `front/package.json`, `front/tsconfig.node.json`, `front/src/main.tsx`.

2.2 Vite, plugin React, router y HTTP

Vite — `front/package.json` `^6.4.3` (lock `6.4.6`) | Capa: frontend/desarrollo y build. Herramienta para servir el entorno de desarrollo y empaquetar la SPA. Docker construye desde `./front`, por lo que esa configuración es la pertinente a dicho build. El maestro recomienda mantener Vite para el flujo existente; técnicamente integra desarrollo/build React. No se halló comparación formal ni motivación histórica original. El servidor rápido de desarrollo no prueba rendimiento en producción. Hay `vite.config.ts`, `vite.config.js` y `vite.forward.config.js` casi duplicados/variantes, y los scripts no seleccionan explícitamente la variante de forwarding: mantenerlas crea riesgo de drift.

Discrepancia de manifiestos: en la raíz también hay `package.json` / `package-lock.json` que declaran Vite `^8.3.8`, frente a `front/` con Vite 6. No debe resumirse como una única versión homogénea: Docker usa el contexto `front/`; definir cuál es la fuente de verdad y documentar la raíz. Evidencia: `front/package.json`, `front/package-lock.json`, `front/vite.config.ts`, `front/vite.config.js`, `front/vite.forward.config.js`, `front/Dockerfile`, `package.json`, `package-lock.json`.

`@vitejs/plugin-react` — `^4.3.4` (lock `4.7.0`) | Capa: frontend/build. Plugin de integración React para Vite, registrado como `react()` en la configuración TS/JS. El repositorio prueba que está activado; el motivo funcional se infiere de esa integración. No se documenta una selección histórica o comparación separada. Es herramienta de desarrollo/build, no runtime. Evidencia: `front/vite.config.ts`, `front/vite.config.js`, `front/package.json`.

React Router DOM — `^7.18.4` (lock `7.18.4`) | Capa: frontend/runtime. Router SPA que asigna rutas URL a páginas/layouts y habilita `Link` / `useNavigate`. `BrowserRouter` envuelve la aplicación; `Routes` y `Route` organizan las vistas y los guards controlan presentación por sesión/rol. Motivo documentado en DT-02: navegación y guards; inferencia: una SPA multipágina necesita mapear URL a vista. No se documenta una comparación histórica. Límites: guards cliente no son autorización del servidor; `CitasAdmin.tsx` existe pero no aparece registrada en `AppRoutes.tsx`, por lo que el archivo no prueba que sea accesible. Evidencia: `front/src/routes/AppRoutes.tsx`, `front/src/components/PrivateRoute.tsx`, `front/src/components/RoleRoute.tsx`, `front/src/layouts/MainLayout.tsx`, `front/src/pages/CitasAdmin.tsx`.

Axios — `^1.7.9` (lock `1.20.0`) | Capa: frontend/runtime, integración API. Cliente HTTP para solicitudes desde el navegador. `src/services/api.ts` configura `VITE_API_URL` o fallback local, agrega Bearer token desde `localStorage` y ante 401 limpia estado local/redirige a login. El maestro lo documenta como cliente centralizado; inferencia técnica: una capa común evita repetir base URL e interceptores. No hay justificación histórica frente a `fetch`. Límites: la URL es configuración de build; `localStorage` es accesible desde la página; borrar el token cliente no lo revoca en servidor. Un comentario heredado menciona `Login.jsx`, aunque la pantalla actual es `Login.tsx` y `allowJs:false`. Evidencia: `front/src/services/api.ts`, `front/src/context/AuthContext.tsx`, `front/src/pages/Login.tsx`, `front/Dockerfile`.

2.3 Iconos, estilos y tipografía

React Icons — `^5.4.0` (lock `5.7.0`) | Capa: frontend/runtime/presentación. Biblioteca que expone iconos como componentes. El uso comprobado es puntual en `NuevaCita` (`FaCalendarAlt`, `FaClock`, `FaUserMd`, `FaCheck`); no demuestra una navegación global terminada. Motivo técnico inferido: reutilizar iconos en esa

pantalla; selección histórica no documentada. Evidencia: `front/src/pages/NuevaCita.tsx`, `front/package.json`.

CSS nativo | Capa: frontend/presentación. Lenguaje de estilos web. `src/index.css` contiene reglas globales, variables, componentes visuales, breakpoints y `prefers-reduced-motion`; se importa desde `main.tsx`. No se observó Tailwind, Bootstrap, MUI, Chakra, Ant Design ni otro framework CSS/UI entre las dependencias/imports revisados. La razón histórica para CSS propio no consta; técnicamente evita un framework adicional y adapta estilos al producto. Límite: reglas globales/nombres de clase sin aislamiento por componente. Hallazgo estático a revisar: `color: var(--ink-600 #060906)` parece sintaxis inválida de `var()` para fallback (normalmente se requiere coma); no se validó en navegador. Evidencia: `front/src/index.css`, `front/src/main.tsx`, `front/package.json`.

Google Fonts — DM Sans y Nunito, sin versión declarada | Capa: frontend/presentación. Fuentes servidas remotamente, asignadas a cuerpo y display respectivamente, con fallbacks locales. Los nombres/roles se ven en CSS; no se encontró motivo escrito ni versión de paquete. Beneficio inferido: sistema tipográfico diferenciado; límite: la descarga inicial depende del servicio/red externa. Evidencia: `front/src/index.css`.

Nginx — `nginx:1.27-alpine` | Capa: frontend/runtime y despliegue. Servidor web/proxy que entrega los archivos compilados de la SPA y dirige `/api/`, `/health` y `/api-docs` a `backend:3000`; `try_files` resuelve rutas de SPA con `index.html`. El maestro y la configuración describen esa responsabilidad; la motivación histórica frente a otros servidores no está registrada. El tag no fija patch/digest; la configuración observada escucha HTTP en 80 y no declara TLS, healthcheck frontend, headers de seguridad o límites. Evidencia: `front/Dockerfile`, `front/nginx.conf`, `docker-compose.yml`.

3. Backend: runtime, lenguaje y API

Node.js — 22, imagen `node:22-alpine` (parche/minor/digest no fijados) | Capa: backend/runtime. Entorno de ejecución de JavaScript. Ejecuta el servidor, scripts npm y APIs nativas `crypto` / `Intl`. El maestro DT-01 justifica mantener el stack existente y señala que el perfil I/O es adecuado al modelo asíncrono; aclara que no acredita una comparación histórica de rendimiento. La motivación original de selección no consta. Beneficio técnico: ecosistema único para API y scripts; límite: tareas CPU/síncronas largas pueden bloquear el event loop; no existe benchmark comparativo con FastAPI. Evidencia: `back/Dockerfile`, `back/server.js`, `back/app/server.js`, documento maestro, DT-01.

JavaScript con módulos ES (ESM) — versión del estándar no declarada | Capa: backend/lenguaje. Lenguaje y formato modular de la API. `package.json` declara `type: module`; archivos de servidor/rutas usan `import` / `export`. Implementa aplicación, rutas, middleware, controladores, modelos, servicios y utilidades. El maestro identifica ESM y registra migración; el código confirma la configuración. No consta motivación histórica adicional. Límite: tipado dinámico; TypeScript es la tecnología del frontend y no el lenguaje ordinario de aplicación backend. Evidencia: `back/package.json`, `back/app/app.js`, `back/app/server.js`, `back/server.js`.

Express — `^4.18.2` (lock `4.22.3`) | Capa: backend/API. Framework HTTP para middleware, rutas y respuestas. Monta CORS, parsers, healthcheck, API `/api`, Swagger, identificación, auditoría y errores. DT-03 documenta organización modular/flexibilidad; la arquitectura específica se infiere de routers y capas. No se encontró comparación histórica formal. No ofrece por sí solo autenticación/autorización, rate limiting ni política segura de CORS. Evidencia: `back/app/app.js`, `back/app/routes/index.js`, `back/app/server.js`, `back/package.json`/lock.

HTTP — versión no declarada | Capa: contrato/transport. Protocolo de solicitudes/respuestas de la API; endpoints usan métodos como GET, POST, PUT, PATCH y DELETE, con healthcheck `/health`. El maestro describe la frontera frontend/API como HTTP/JSON; no hay motivo histórico del protocolo documentado. El código

revisado no fija versión de HTTP ni demuestra TLS para todos los entornos. Evidencia: `back/app/server.js`, `back/app/routes/`, `back/app/config/swagger.js`, `front/nginx.conf`.

JSON — formato sin versión declarada | Capa: contrato/API. Formato de cuerpos entrantes y salientes. Express parsea JSON; las respuestas comunes usan estructuras como `success`, `message`, `data`, `code` y `details`, aunque no todos los handlers comparten idéntico contrato. La documentación maestra registra HTTP/JSON; razón histórica de formato no localizada. Evidencia: `back/app/app.js`, `back/app/utils/response.js`, `back/app/config/swagger.js`.

Aclaración fundamental: React, JavaScript y FastAPI

React es una biblioteca de **frontend** ejecutada en el navegador; no reemplaza Node/Express ni es un framework Python. El backend actual está escrito en **JavaScript ESM** y corre en **Node.js** con **Express**. **FastAPI** es un framework de backend para **Python**: no traduce código JavaScript a Python ni migra automáticamente Express. Un cambio a FastAPI exigiría reimplementar el backend, sus rutas, validaciones, reglas, persistencia, autenticación y pruebas en Python. El material auditado no muestra FastAPI instalado/usado ni una decisión adoptada para migrar. No hay benchmark comparativo que permita afirmar que Express o FastAPI sea más rápido en este proyecto.

4. Persistencia: Prisma, PostgreSQL y SQL

PostgreSQL — 16, `postgres:16-alpine` | Capa: datos. Motor relacional que guarda las ocho entidades actuales y aplica claves, restricciones, índices y transacciones. DT-04 documenta su adecuación por relaciones, integridad y concurrencia; la estructura confirma ese uso. No hay comparación formal de motores. Compose fija major 16, pero no hay servidor inspeccionado ni patch/digest efectivo; `back/Readme.md` menciona 12+ como requisito genérico, menos específico que Compose. Evidencia: `docker-compose.yml`, `back/prisma/schema.prisma`, `back/prisma/migrations/`, maestro, DT-04.

Prisma ORM y esquema declarativo — `^7.10.1` (lock `7.10.1`) | Capa: datos/diseño. ORM y representación declarativa de modelos/relaciones. El schema actual define ocho modelos; sus tipos, relaciones, claves y parte de unicidad alimentan el cliente. La bitácora registra migración desde Sequelize y motivos de mantener los contratos/restricciones; DT-05 enumera esquema, CRUD y transacciones. No aparece una comparación formal contemporánea completa. Ventaja: centraliza modelo y operaciones comunes. Límites importantes: el schema no refleja íntegramente CHECKs, triggers ni el índice único parcial; no garantiza consultas óptimas ni elimina SQL. Evidencia: `back/prisma/schema.prisma`, `back/app/models/`, `BITACORA_MODERNIZACION_PRISMA_Y_CITAS.md`, maestro, DT-05.

Prisma Client — `^7.10.1` (lock `7.10.1`) | Capa: datos/runtime. Cliente generado que proporciona operaciones CRUD, relaciones, transacciones y `$queryRaw`. Se instancia en un singleton y lo usan modelos de usuarios, mascotas, citas, disponibilidad e historias. La bitácora documenta contratos y transacciones; inferencia: comparte acceso configurado a la BD. Depende de regeneración tras cambios de schema y no representa automáticamente objetos SQL personalizados. Evidencia: `back/app/config/prisma.js`, `back/app/models/`, `back/prisma/schema.prisma`.

Prisma CLI y Prisma Migrate — `^7.10.1` (lock `7.10.1`) | Capa: datos/desarrollo-operación. CLI para generar cliente, estado, baseline, migraciones y seed. `prisma.config.ts` fija rutas/configuración. Historial vigente: baseline `0_init` de siete tablas originales y migración posterior para `cita_cancelacion`, que completa ocho modelos/tablas actuales. La bitácora documenta su incorporación y preservación del baseline. Advertencia: `db:reset` es destructivo; no se ejecutó. Registrar baseline como aplicado no ejecuta su SQL. Evidencia: `back/prisma.config.ts`, `back/prisma/migrations/`, `back/package.json`, `back/Readme.md`.

`@prisma/adapter-pg` — `^7.10.1` (lock `7.10.1`) | **Capa: datos/runtime.** Adaptador que conecta Prisma 7 con driver PostgreSQL a través de `connectionString`. Su uso con Prisma/PG está registrado; la elección comparativa entre adapters no. Evidencia: `back/app/config/prisma.js`, `back/package.json`/lock y bitácora de modernización.

`pg` (node-postgres) — `^8.23.0` (lock `8.23.1`) | **Capa: datos/runtime y herramientas de prueba.** Driver PostgreSQL. El código lo conecta indirectamente mediante adapter-pg; las herramientas globales de prueba también lo usan para crear/terminar/eliminar la base de integración. No se encontró selección histórica o benchmark frente a otros drivers. No se observa uso directo en los modelos ordinarios de aplicación. Evidencia: `back/app/config/prisma.js`, `back/tests/setup/globalSetup.js`, `back/tests/setup/globalTeardown.js`, `back/package.json`.

SQL y PL/pgSQL de PostgreSQL — versión no declarada | Capa: datos/integridad y concurrencia. Lenguaje para migraciones, restricciones, triggers y operaciones especializadas. SQL de migración crea CHECKs, trigger functions, índice único parcial `uq_cita_disponibilidad_activa` y tabla `cita_cancelacion`; modelos ejecutan SQL etiquetado parametrizado con `FOR UPDATE` / `SKIP LOCKED` para reservas/procesamiento. DT-05/DT-07 explica que se conserva porque Prisma no expresa por completo estas reglas. Beneficio: control de invariantes y bloqueos; límite: migraciones y schema deben sincronizarse, parámetros dinámicos/identificadores requieren cuidado y entrada externa nunca debe concatenarse como SQL. Evidencia: `back/prisma/migrations/0_init/migration.sql`, `back/prisma/migrations/20261110190000_cita_cancelacion_audit/migration.sql`, `back/app/models/cita.model.js`, `back/app/models/historia.model.js`, `back/app/utils/sqlSeguro.js`.

Aclaración sobre Prisma/PostgreSQL: Prisma no sustituye a PostgreSQL ni convierte el proyecto en portable a cualquier motor. Prisma modela y facilita CRUD/transacciones; PostgreSQL aplica el esquema y ejecuta SQL/PLpgSQL particular. Los triggers, CHECKs, índice parcial y bloqueos deben mantenerse coherentes con el schema y probarse conjuntamente. La referencia heredada a «siete modelos» puede describir el baseline inicial, pero no el estado actual: varias documentaciones antiguas quedaron incompletas frente a la migración `cita_cancelacion`.

5. Seguridad, validación, sesiones, logs y fechas

JWT y `jsonwebtoken` — `^9.0.3` (lock `9.0.3`) | Capa: seguridad/backend. JWT firmado HS256 para autenticación Bearer. El backend verifica algoritmo/firma/expiración y vuelve a consultar usuario/rol en BD; usa identidad vigente de BD, no los headers heredados. DT-06 documenta sustitución de mock/headers manipulables. Inferencia: validar firma y releer rol evita confiar directamente en identidad enviada por cliente. No hay evidencia de comparación formal con sesiones. Límites: no se encontró revocación inmediata ni rate limiting de login; el logout limpia cliente pero no invalida token ya emitido; la protección depende de un `JWT_SECRET` sólido. `jwt.js` usa fallback 8h mientras `.env.example` dice 10h; depende de entorno. Swagger heredado describe otro esquema. Evidencia: `back/app/utils/jwt.js`, `back/app/middlewares/identifyUser.js`, `back/.env.example`, `docker-compose.yml`.

bcrypt — `^6.0.0` (lock `6.0.0`) | Capa: seguridad/credenciales. Hash de contraseñas nuevas; utiliza 12 rondas predeterminadas y permite configuración documentada entre 10 y 15. Login correcto puede elevar hashes válidos a la configuración vigente. El maestro documenta el cambio de hash actual a bcrypt. Inferencia: salt y coste configurable dificultan comparación directa y permiten elevar el costo. No hay benchmark frente a otras funciones. Código limita contraseña a 72 bytes y schema mínimo de cuatro caracteres; no debe confundirse esa restricción con una política robusta de longitud. Evidencia: `back/app/utils/password.js`, `back/app/services/auth.service.js`, `back/app/schemas/auth.schema.js`, maestro DT-06.

scrypt heredado — crypto nativo de Node, sin paquete independiente | Capa: seguridad/compatibilidad. Verificación solamente de hashes históricos con parámetros/formato permitidos; no genera hashes nuevos. Login válido permite transición/rehash a bcrypt. La compatibilidad está documentada; la razón original de adoptar scrypt no está registrada. No tratarlo como mecanismo actual de escritura. Evidencia: `back/app/utils/password.js`, `back/app/services/auth.service.js`, maestro DT-06.

Joi — `^17.13.6` (lock `17.13.9`) | Capa: backend/seguridad/validación. Biblioteca de esquemas para validar auth, usuarios, mascotas, citas, disponibilidad e historias. Middleware aplica `abortEarly:false`, `stripUnknown:true`, `convert:true`, normaliza mensajes y coloca datos validados en la solicitud. El maestro registra validación server-side. Beneficio: contratos de entrada centralizados; límites: forma válida no equivale a autorización, reglas de negocio o integridad BD; `stripUnknown` descarta campos no declarados; validar fecha ISO no basta para determinar futuro en zona de negocio. Evidencia: `back/app/middlewares/validate.js`, `back/app/schemas/`, maestro DT-06.

CORS y `cors` — `^2.8.5` (lock `2.8.6`) | Capa: seguridad/backend. Middleware que establece permisos de acceso desde navegadores según `CORS_ORIGIN`, lista separada por comas y comparación literal; declara métodos/headers y `credentials:true`. La documentación aconseja orígenes exactos; no se encontró motivo histórico para valores específicos. CORS no autentica ni detiene clientes no navegador. Hallazgo importante: Compose usa default `*`, pero el código lo compara literalmente, no implementa wildcard. `.env.example` también recomienda `*`, mientras el documento maestro aconseja lista exacta; corregir/documentar. Evidencia: `back/app/app.js`, `back/.env.example`, Compose, README.

Pino — `^9.14.0` (lock `9.14.0`) | Capa: backend/seguridad/observabilidad. Logger estructurado con nivel por entorno y redacción de claves sensibles. El maestro registra esa política; técnicamente concentra eventos y reduce exposición accidental. No consta por qué se eligió frente a otro logger. `auditLog.js` emite eventos informativos solo fuera de producción; el nivel production `error` no es ledger ni auditoría persistente general. Evidencia: `back/app/utils/logger.js`, `back/app/middlewares/errorHandler.js`, `back/app/middlewares/auditLog.js`.

Normalización de entrada — middleware propio, sin versión | Capa: backend/validación/seguridad. Recorta espacios en strings de forma recursiva, excluye contraseñas/tokens/autorización y aplica Joi después; estandariza errores. El maestro describe normalización; inferencia: reduce inconsistencias por espacios accidentales. No hay decisión histórica comparativa. No es sanitizador universal ni reemplaza validación, autorización, escaping HTML o parametrización SQL. Evidencia: `back/app/middlewares/validate.js`, `back/app/schemas/`.

SQL seguro: Prisma tagged raw SQL y helpers propios — Prisma `^7.10.1`; helper sin versión | Capa: seguridad/datos. Las plantillas etiquetadas enlazan valores; `sqlSeguro.js` valida identificadores enteros, límites y timestamps. Comentarios prohíben variantes Unsafe/concatenación con datos externos; la revisión reportada encontró `$queryRaw` parametrizado en modelos y no halló llamadas Unsafe en aplicación. El maestro documenta este uso por bloqueos/consultas especiales. Los identificadores dinámicos requieren allowlist; la seguridad depende de mantener disciplina en nuevas consultas. Evidencia: `back/app/utils/sqlSeguro.js`, `back/app/models/cita.model.js`, `back/app/models/historia.model.js`.

Fecha y hora: `Date` / `Intl.DateTimeFormat` nativos — API Node, sin paquete | Capa: backend/dominio/validación. El código aplica `America/Bogota` para el reloj de negocio y maneja ciertos valores ISO. El maestro documenta zona para franjas/citas; inferencia: especificar zona evita depender del host. No existe selección histórica de una biblioteca de fechas. Joi valida formatos en otra capa; `Date`/ISO y zona local requieren consistencia al intercambiar strings/objetos `Date`. Evidencia: `back/app/utils/fechaHora.js`, `back/app/utils/sqlSeguro.js`, `back/app/schemas/cita.schema.js`, `back/app/schemas/disponibilidad.schema.js`.

dotenv — `^16.3.1` (lock `16.6.1`) | Capa: configuración/runtime y tooling. Carga `.env`, `.env.test` y variables que parametrizan JWT, CORS, bcrypt y DB. Los ejemplos lo justifican como configuración por entorno;

no es gestor/validador de secretos. Compose puede suministrar valores inseguros por defecto. Un `.env.test` tampoco garantiza base local: variables exportadas pueden prevalecer; revisar host y valores efectivos antes de tests. Evidencia: `back/app/server.js`, `back/app/config/prisma.js`, `back/prisma.config.ts`, `back/tests/setup/setupEnv.js`, `.env.example`, `back/.env.example`, `docker-compose.yml`.

`crypto` nativo de Node — API runtime, sin paquete | Capa: seguridad. Provee `scrypt`, comparación de tiempo constante y aleatoriedad empleadas por `password.js`; no es una dependencia npm ni reemplaza bcrypt actual. Motivo inferido: ya está disponible en Node. No consta decisión histórica independiente. Evidencia: `back/app/utils/password.js`, Dockerfile backend.

6. Pruebas, calidad y documentación del contrato

Vitest — `4.1.12` | Capa: pruebas/backend. Runner configurado para `tests/**/*.test.js`, Node y suites unitarias/integración. Configuración usa `forks`, un worker y sin paralelismo entre archivos porque comparten BD. La bitácora documenta migración desde Jest para compatibilidad Node/ESM y separación de DB; no acredita comparación formal de runners. Actualmente hay 23 archivos (12 integración y 11 unitarios según auditoría), que prueban varias capas; su presencia no certifica una ejecución actual. La auditoría consolidada no ejecutó tests. Evidencia: `back/vitest.config.js`, `back/tests/`, `back/package.json`, `BITACORA_MIGRACION_VITEST.md`.

Supertest — `^7.2.2` (lock `7.3.2`) | Capa: pruebas/backend HTTP. Cliente de pruebas que envía solicitudes al objeto Express y verifica status/cuerpos/rutas con integración a persistencia. La bitácora indica que se conservó tras migrar a Vitest; motivo original no registrado. No prueba la SPA en navegador ni conformidad OpenAPI automáticamente. Evidencia: `back/tests/helpers/auth.helper.js`, `back/tests/integration/`, package/lock.

`@vitest/coverage-v8` — `4.1.12` | Capa: pruebas/cobertura. Proveedor V8 para informes text, HTML y LCOV de archivos de aplicación incluidos. La bitácora registra instalación para cobertura; motivo de preferir V8 no documentado. Config excluye `app.js`, `server.js` y `swagger.js`, y no fija umbral mínimo. Cobertura tampoco demuestra calidad de assertions ni cumplimiento del contrato. Evidencia: `back/vitest.config.js`, `back/package.json`, `BITACORA_MIGRACION_VITEST.md`.

`cross-env` — `^10.1.0` (lock `10.1.0`) | Capa: tooling de pruebas. Scripts lo usan para fijar `NODE_ENV=test`; inferencia: centraliza esa variable para comandos npm. Motivo histórico no registrado. Evidencia: `back/package.json`.

Type-check frontend — TypeScript y `@types` ya inventariados | Capa: calidad/build frontend. `type-check` y `build` comprueban/compilan tipos, no son framework de pruebas. No se encontraron archivos `*.test.*` / `*.spec.*`, framework frontend/E2E (Jest/Vitest/Playwright/Cypress) ni automatización de navegador en `front/`. Evidencia: `front/package.json`, árbol `front/src/`, maestro DT-02.

OpenAPI — 3.0.0 declarado; `swagger-jsdoc` `^6.2.8` (lock `6.3.0`); `swagger-ui-express` `^5.0.0` (lock `5.0.1`) | Capa: documentación/API. Objeto de especificación manual en `back/app/config/swagger.js`, servido por Express en `/api-docs`. `apis: []` indica que no escanea anotaciones de rutas; paths/components son escritos manualmente. `swagger-jsdoc` ensambla definición, UI la presenta. No se documentó motivo histórico de elegir formato/versiones. Límite crítico: no se encontraron tests que validen responses contra OpenAPI; Swagger conserva `x-user-id`, `x-user-role` y `mock-token` frente a JWT Bearer vigente. Tratar como documentación desactualizada hasta alinear esquema de seguridad, ejemplos y comportamiento real. Evidencia: `back/app/config/swagger.js`, `back/app/app.js`, `docs/back/API.md`.

npm — versión no fijada | Capa: desarrollo/build/operación. Instala dependencias con `npm ci` y ejecuta scripts de dev, build, tests, migración, seed y start. El lockfile es versión 3; la versión npm deriva del entorno Node

y no está declarada directamente. Uso comprobado, sin motivo histórico comparativo frente a otros gestores. Evidencia: `back/package.json`, `front/package.json`, Dockerfiles y lockfiles.

`tsx` — `^4.23.15` (lock `4.23.15`) | Capa: backend/scripts de operación. Ejecuta servidor en scripts `start` / `dev`, scripts de migración/baseline/contraseñas y seed declarado por Prisma. Es dependencia runtime declarada aunque el backend de aplicación sea JS; no inferir uso de TypeScript en el backend por su nombre. Motivo de selección no documentado; el uso se infiere de scripts. Evidencia: `back/package.json`, `back/prisma.config.ts`, `back/prisma/seed.js`.

nodemon — `^3.0.1` (lock `3.1.14`) | Capa: desarrollo backend. Vigila `server.js` / `app` y reinicia `tsx server.js` al cambiar archivos en `npm run dev`. Herramienta dev, no runtime productivo. No se halló razón histórica de selección. Evidencia: `back/package.json`.

ESLint — versión no declarada y dependencia local no declarada | Capa: calidad frontend intencionada, estado no verificado. El script frontend invoca `eslint .`, pero `package.json` no declara ESLint como dependencia. No se ejecutó lint y no se comprobó binario global. No afirmar que funciona ni que está completamente ausente del entorno. Evidencia: `front/package.json`.

7. Contenedores, infraestructura y operación

Docker — versión no declarada | Capa: despliegue/desarrollo. Construye imágenes por Dockerfiles; frontend usa build multi-stage Node→Nginx y backend Node. Documentado para entorno local/demo y tests; inferencia: empaqueta servicios con sus entornos. No hay decisión comparativa histórica. Tags sin digest, TLS o configuración de producción. Evidencia: `back/Dockerfile`, `front/Dockerfile`, `docker-compose.yml`, README.

Docker Compose — README indica v2; YAML no fija versión | Capa: despliegue local/test. Coordina `db`, `backend`, `frontend`; Compose de test define `db-test` y `tests`, espera healthcheck y usa tmpfs para datos temporales. Es configuración documentada para desarrollo/QA, no despliegue de producción. Beneficio inferido: servicios, dependencias y variables declarativas; alternativa histórica no registrada. Compose de test permite operaciones destructivas en base validada; no se ejecutó. Evidencia: `docker-compose.yml`, `docker-compose.test.yml`, `back/tests/setup/testDbConfig.js`.

Healthchecks de Compose | Capa: operación. PostgreSQL usa `pg_isready`; backend consulta `/health`; frontend depende de backend healthy y backend de DB healthy. No hay healthcheck frontend, aunque README dice que los tres servicios lo tienen. Es una inconsistencia documental; tampoco hay monitorización/alertas declaradas. Evidencia: Compose, `back/app/app.js`, README.

`docker-entrypoint.sh` y `/bin/sh` — sin versión | Capa: despliegue/backend. Script de arranque que espera PostgreSQL, ejecuta migraciones, opcionalmente seed y luego `exec` del comando de servidor. Hace cambios de base al inicio; no coordina migraciones de múltiples réplicas. Seed exige `DB_SEED=true` y `NODE_ENV` distinto de `production`; Compose pone production por defecto, por lo que DB_SEED aislado no basta. Compose de test anula el entrypoint. Evidencia: `back/docker-entrypoint.sh`, `back/Dockerfile`, `docker-compose.yml`, `docker-compose.test.yml`.

Node para build frontend y runtime backend — Node 22 `node:22-alpine` | Capa: build/despliegue. Node compila frontend en etapa de build y ejecuta API; la imagen final frontend es Nginx. Tags fijan línea mayor/distribución, no parche. Compatibilidad de dependencias debe revisarse al fijar imagen reproducible; no se hizo build. Evidencia: Dockerfiles, manifiestos.

PostgreSQL contenedorizado — `postgres:16-alpine` | Capa: datos/despliegue. Servicio persistente local en `postgres_data`; test usa tmpfs. Sin backup automatizado ni puerto de DB publicado en Compose principal según auditoría. Compose y maestro coinciden en major 16; no confirma servidor actualmente en ejecución. Evidencia: Compose principal, Compose de test.

Nginx frontend — `nginx:1.27-alpine` | Capa: runtime/despliegue. Además de servir la SPA, proxya API dentro de red Compose. Tag sin patch/digest. Configuración observada no declara HTTPS/TLS y escucha en 80; no presentar como arquitectura productiva lista. Evidencia: `front/nginx.conf`, `front/Dockerfile`.

Archivos `.env` /dotenv — formato sin versión; dotenv backend ya descrito | Capa: configuración/despliegue. Compose interpola variables y backend las lee. `.dockerignore` excluye `.env`; `.env.example` provee nombres/plantillas. No es bóveda de secretos. Evidencia: `.env.example`, `back/.env.example`, ignores, `back/.dockerignore`, `front/.dockerignore`, Compose.

8. Elementos observados, pero no elegidos como tecnología

- **FastAPI** aparece como alternativa mencionada en el material comparativo, no como dependencia ni framework presente. Es un framework Python y su adopción sería una reimplementación, no traducción automática. No se eligió según la evidencia disponible.
- **Sequelize** aparece como antecedente de migración a Prisma; no se inventaría como ORM vigente. La bitácora documenta transición, pero no necesariamente todo el historial técnico.
- **Tailwind, Bootstrap, MUI, Chakra, Ant Design** no se encontraron en dependencias/imports revisados. Esto no prueba que jamás se hayan evaluado; solo que no se observan como stack actual.
- **Jest** se menciona como runner anterior en bitácora de migración; el runner actual observado es Vitest.
- **Playwright, Cypress y otros E2E/browser** no se encontraron configurados en frontend.
- **Vite forward config** es una variante de configuración, no tecnología de runtime. Mantener una sola fuente clara y documentar forwarding.
- **ESLint** está invocado por un script, pero no declarado como dependencia; su disponibilidad no se verificó.
- **CitasAdmin** es un archivo fuente no una ruta registrada; no inferir una funcionalidad desplegada o accesible solo por su existencia.

9. Tecnologías futuras/pendientes: no hay elecciones que inventar

La auditoría y el documento maestro dejan necesidades de mejora, pero no establecen necesariamente tecnologías seleccionadas para resolverlas. Por tanto, esta sección registra **requisitos pendientes**, no recomendaciones de librerías adoptadas:

1. **Pruebas frontend y E2E:** incorporar una estrategia automatizada que cubra componentes/flujo de navegador; framework concreto no documentado como elección.
2. **Contrato API:** alinear Swagger/OpenAPI con Bearer JWT y respuestas actuales y automatizar conformidad del contrato; herramienta para ello no elegida.
3. **Seguridad de autenticación:** evaluar rate limiting de login y mecanismo de revocación/invalidez de tokens; no se encontró tecnología escogida.
4. **Gestión de secretos y producción:** retirar fallbacks/placeholders, definir secretos gestionados, HTTPS, backups y operación/observabilidad; proveedor/plataforma de despliegue no definido.
5. **CORS:** establecer allowlist por entorno coherente con comparación literal; la política exacta no debe inferirse del `*` actual.
6. **Calidad/build frontend:** aclarar manifiestos Vite 6 vs raíz Vite 8, elegir configuración única, verificar ESLint y corregir/validar regla CSS sospechosa.

7. **Integridad/persistencia:** mantener schema, migraciones, triggers y SQL sincronizados; documentar procedimiento seguro de migración y prevenir divergencias.
8. **Reproducibilidad:** fijar tags/digests de imágenes y versiones de runtimes apropiadas; no se ha registrado una versión futura específica.

No se presenta ninguno de esos puntos como decisión ya adoptada ni se asigna nombre de producto/librería sin evidencia.

10. Riesgos, contradicciones y verificaciones prioritarias

Prioridad	Hallazgo y por qué importa	Fuente a contrastar
Alta	Compose permite `DB_PASSWORD=postgres` y `JWT_SECRET=change-this-secret-in-production` por defecto; retirar defaults de demo antes de producción y gestionar secretos.	`docker-compose.yml`, `back/.env.example`, maestro
Alta	`CORS_ORIGIN=*` se compara literalmente y no es wildcard; además README dice que backend no usa la variable, aunque `app.js` sí.	`back/app/app.js`, `docker-compose.yml`, `Readme.md`
Alta	JWT no tiene revocación inmediata hallada ni rate limit de login; cliente elimina token sin invalidarlo en server.	`back/app/utils/jwt.js`, `back/app/middlewares/identifyUser.js`, maestro
Alta	Swagger describe headers/mock-token heredados en lugar de Bearer JWT; contrato manual sin test de conformidad.	`back/app/config/swagger.js`, `API.md`, `identifyUser.js`
Alta	Setup/teardown de pruebas crea y elimina base; guard exige `_test` y distinto de `DB_NAME`, pero no garantiza que host sea local. Revisar host/credenciales/`DB_NAME_TEST` efectivos antes de test.	`back/tests/setup/testDbConfig.js`, `globalSetup.js`, `globalTeardown.js`
Media	`.env.example` expone `JWT_EXPIRES_IN=10h`; fallback de código es 8h.	`back/.env.example`, `back/app/utils/jwt.js`
Media	Vite de `front/` 6.x vs raíz 8.x; Docker usa `front/`. Configs Vite duplicadas/forward pueden derivar.	`front/package.json`, `package.json`, configs Vite
Media	README afirma healthcheck de tres servicios; Compose principal solo define DB y backend.	`docker-compose.yml`, `Readme.md`
Media	README y documentos antiguos describen siete modelos/tablas, mientras estado actual suma `cita_cancelacion` y ocho modelos. Siete corresponde a baseline histórico inicial.	`back/prisma/schema.prisma`, migraciones, `Readme.md`, maestro

Prioridad	Hallazgo y por qué importa	Fuente a contrastar
Media	Algunas partes de README describen scrypt actual; el código/documento maestro indican bcrypt nuevo y scrypt solo verificación histórica.	`back/app/utils/password.js`, README, maestro
Media	Frontend no tiene suite tests/E2E; build/type-check no equivalen a pruebas de UI.	`front/package.json`, árbol `front/`
Media	`CitasAdmin.tsx` no está registrada; confirmar si se pretende exponerla.	`front/src/pages/CitasAdmin.tsx`, `front/src/routes/AppRoutes.tsx`
Baja	CSS `var(--ink-600 #060906)` parece inválida; hallazgo estático, no validado en browser.	`front/src/index.css`
Baja	Comentario del servicio Axios menciona `Login.jsx` heredado, pero existe `Login.tsx`, y `allowJs:false`.	`front/src/services/api.ts`, `front/src/pages/Login.tsx`, `tsconfig.json`
Baja	Script `lint` invoca eslint sin dependencia declarada; no se comprobó instalación global ni ejecución.	`front/package.json`
Baja	Imágenes Docker usan tags, no digests; no se inspeccionó runtime desplegado.	Dockerfiles y Compose

11. Relación de responsabilidades y arquitectura modular de tres capas

La solución puede entenderse como tres capas principales con responsabilidades separadas, más seguridad/calidad/operación transversales:

1. **Presentación (frontend):** React/React DOM renderizan la SPA; TypeScript aporta tipos de desarrollo; React Router enlaza URL con páginas/layouts; Axios comunica con la API; CSS y fuentes componen la presentación. Nginx entrega el build y proxya rutas API. El cliente guía la experiencia por rol, pero nunca decide por sí solo la autorización real.
2. **Aplicación/API (backend):** Node.js ejecuta JavaScript ESM; Express compone middlewares y rutas HTTP/JSON; Joi valida entradas; servicios/controladores implementan reglas y respuestas; JWT identifica al usuario y el middleware consulta rol vigente. La API es la frontera de confianza y debe validar/autorización en servidor.
3. **Persistencia (datos):** PostgreSQL conserva el modelo relacional e invariantes. Prisma schema/Client y migraciones soportan modelo declarativo y CRUD; SQL/PLpgSQL complementa objetos no expresados plenamente por Prisma y operaciones de bloqueo/concurrencia.

Transversales: seguridad incluye hashing, tokens, CORS, redacción y parametrización SQL; pruebas backend con Vitest/Supertest ejercitan módulos, HTTP y BD; OpenAPI documenta el contrato aunque hoy requiere

alineación; Docker/Compose organizan entorno local/test. No se atribuye producción lista al stack: los documentos describen desarrollo/demo y dejan pendientes secretos, TLS, backups, operación y estrategia de despliegue.

12. Alcance y limitaciones de la consolidación

Este documento combina inventarios estáticos y hallazgos aportados por auditorías de varios dominios. Las rutas enlazadas son relativas al repositorio, partiendo de `docs/back/`. Los números de versión distinguen manifiesto/rango y lock cuando ambos estaban disponibles; versiones de imagen se citan por tag. No se validó que los servicios estén ejecutándose ni que cada script/estilo/configuración funcione en runtime. Los resultados históricos de bitácoras no se presentan como ejecuciones actuales. Antes de tomar decisiones de mantenimiento conviene contrastar cada enlace con el checkout vigente.